

CLI Agent — Complete Project Guide

Architecture, Design Patterns, Memory Systems, Security, and Implementation
Phases for an AI-Powered CLI Agent built with TypeScript/Node.js

Index

Phase 0 — Project Foundation & Setup

- 0.1 Initialize Project Structure
- 0.2 Dev Tooling & Configuration
- 0.3 Core Type Definitions

Phase 1 — CLI Interface & User Experience

- 1.1 Build the Chat Terminal UI
- 1.2 Streaming & Real-Time Output
- 1.3 Input Handling & Command Parsing

Phase 2 — Agentic Loop (ReAct Pattern)

- 2.1 Agent Loop Engine
- 2.2 LLM Provider Abstraction (Adapter Pattern)
- 2.3 Response Parsing & Structured Output

Phase 3 — Tool Registry & Tool Executor

- 3.1 Tool Registry (Factory / Service Locator Pattern)
- 3.2 Tool Executor (Command Pattern)
- 3.3 Security & Sandboxing
- 3.4 Built-in & Custom Tools

Phase 4 — Context Building & Retrieval

- 4.1 AST Parsing (TypeScript Compiler API)

- 4.2 Embeddings Generation
- 4.3 Vector Database Setup
- 4.4 Incremental Index Updates
- 4.5 Query-Time Retrieval & Ranking

Phase 5 — Memory Architecture

- 5.1 Working Memory (In-Memory)
- 5.2 Episodic Memory (SQLite)
- 5.3 Procedural Memory (Config File)
- 5.4 Memory Integration & Lifecycle

Phase 6 — Guardrails & Error Handling

- 6.1 Pre-Call Validation
- 6.2 Post-Call Validation
- 6.3 Output Guardrails
- 6.4 Recovery & Fallback Strategies

Phase 7 — Edge Cases & Their Handling

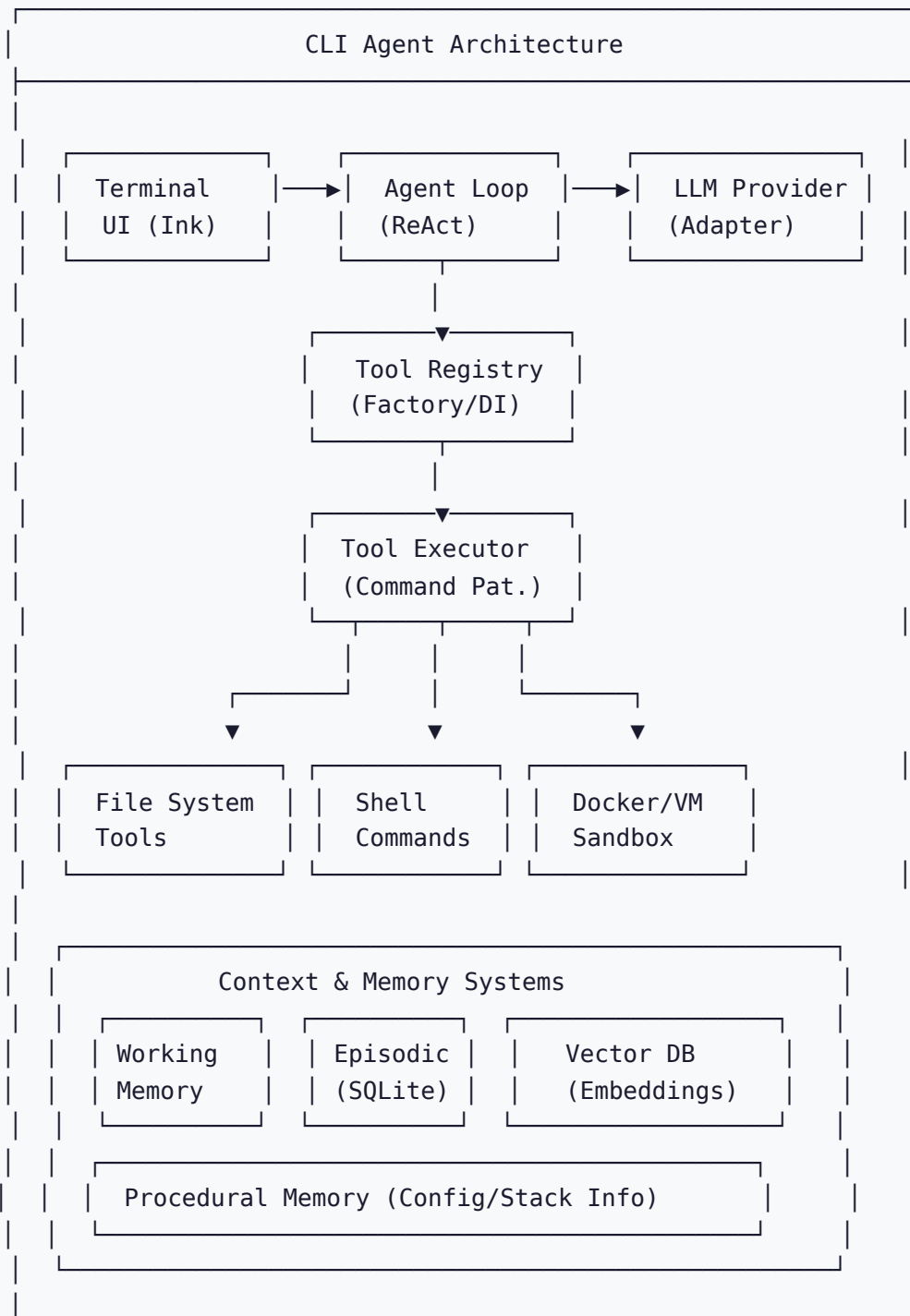
- 7.1 Infinite Loops / Runaway Processes
- 7.2 Excessive Output from Tools
- 7.3 Malicious Code Execution
- 7.4 Stale Embeddings / File Changes
- 7.5 LLM Returns Irrelevant Responses
- 7.6 User Input Variability (Synonyms & Fuzzy Queries)
- 7.7 Large Conversation History

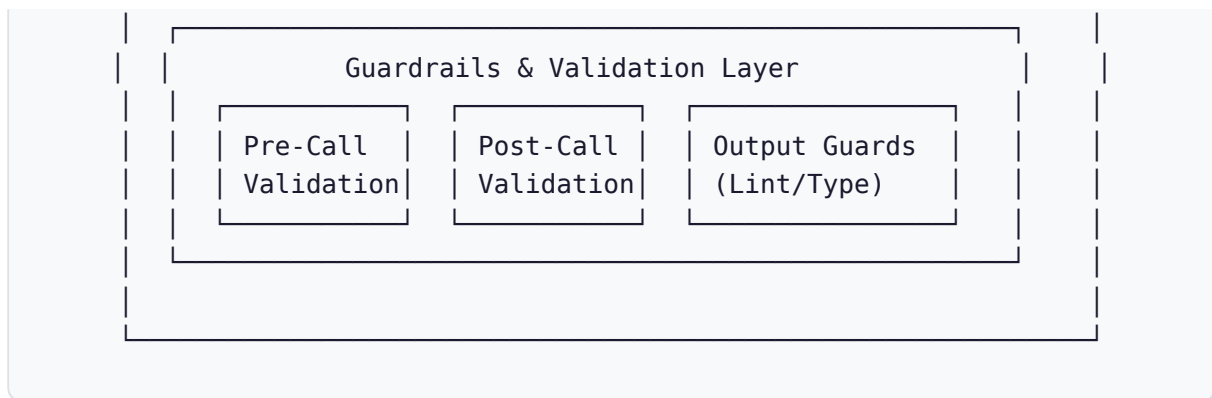
Phase 8 — Testing, Deployment & Maintenance

- 8.1 Testing Strategy
- 8.2 Distribution & Updates (npm)
- 8.3 Logging & Observability
- 8.4 Future Upgrades (Docker Sandbox, Multi-Model)

Reading Guide: Each phase is self-contained with implementation steps, design patterns, code snippets, resource links, and edge-case notes. Follow phases sequentially — each builds on the previous.

System Architecture Overview





Component	Design Pattern	Library / Tool	Purpose
CLI Interface	Component Composition	Ink (React for CLI)	Rich terminal UI with colors, input fields, real-time output
Agent Loop	ReAct Pattern	Custom implementation	Alternate between Reasoning and Acting (Tool Calls)
LLM Provider	Adapter Pattern	OpenAI / Anthropic SDKs	Unified interface for multiple LLM providers
Tool Registry	Factory / Service Locator	Custom	Register, discover, and resolve tools dynamically
Tool Executor	Command Pattern	Node.js <code>child_process</code>	Encapsulate tool actions as command objects
AST Parsing	Structural Analysis	TypeScript Compiler API	Extract functions, classes, signatures from source code
Embeddings	Vector Encoding	sentence-transformers / OpenAI Embeddings	Convert code chunks into semantic vectors
Vector Storage	Similarity Search	FAISS (local) / sqlite-vec	Store and query embeddings locally

Episodic Memory	Structured Logging	SQLite via better-sqlite3	Store conversation history, key decisions, tool call results
Procedural Memory	Config File	JSON / YAML	Store stack version, framework config, project metadata
Dependency Injection	DI Container	tsyringe (optional)	Loose coupling between components

Phase 0 — Project Foundation & Setup

0.1

Initialize Project Structure

```
cli-agent/
├── src/
│   ├── cli/                # Terminal UI & input handling
│   │   ├── App.tsx         # Ink app entry
│   │   ├── InputBar.tsx    # User input component
│   │   └── OutputArea.tsx  # Streaming output display
│   ├── agent/              # Agentic loop & LLM interaction
│   │   ├── AgentLoop.ts    # ReAct loop engine
│   │   ├── providers/      # LLM provider adapters
│   │   │   ├── IProvider.ts
│   │   │   ├── OpenAIProvider.ts
│   │   │   └── AnthropicProvider.ts
│   │   └── parsers/        # LLM response parsers
│   ├── tools/              # Tool system
│   │   ├── registry/       # Tool registry (factory)
│   │   │   └── ToolRegistry.ts
│   │   ├── executor/       # Tool executor (command pattern)
│   │   │   └── ToolExecutor.ts
│   │   └── built-in/       # Built-in tool implementations
│   │       └── ReadFileTool.ts
```


- **Build:** `tsup` or `tsc` for bundling
- **Linting:** ESLint + Prettier
- **Testing:** Vitest (fast, compatible with TS)
- **Package Manager:** pnpm (faster, disk-efficient)

Resources: [tsup docs](#) — [Vitest docs](#) — [pnpm docs](#)

0.3

Core Type Definitions

```
// src/types/index.ts

/** A tool that can be registered and executed */
export interface Tool {
  name: string;
  description: string;
  parameters: Record<string, ParameterSchema>;
  execute(args: Record<string, unknown>): Promise<ToolResult>;
}

export interface ParameterSchema {
  type: 'string' | 'number' | 'boolean' | 'array' | 'object';
  description: string;
  required?: boolean;
}

export interface ToolResult {
  success: boolean;
  data?: unknown;
  error?: string;
  truncated?: boolean; // true if output was capped
}

/** LLM provider abstraction */
export interface ILLMProvider {
```

```

    chat(messages: Message[], options?: ChatOptions): Promise<Chat>
    chatStream(messages: Message[], options?: ChatOptions): AsyncI
  }

export interface Message {
  role: 'system' | 'user' | 'assistant' | 'tool';
  content: string;
  toolCallId?: string;
}

export interface ChatOptions {
  temperature?: number;
  maxTokens?: number;
  tools?: ToolDefinition[];
}

export interface ToolDefinition {
  name: string;
  description: string;
  inputSchema: Record<string, unknown>;
}

/** Memory entry for episodic storage */
export interface EpisodicEntry {
  id: string;
  timestamp: number;
  type: 'tool_call' | 'decision' | 'observation' | 'error';
  summary: string;
  details?: Record<string, unknown>;
}

/** Project configuration (procedural memory) */
export interface ProjectConfig {
  stack: Record<string, string>; // "typescript": "5.4", "next
  lastUpdated: number;
  constraints: string[];
}

```

Phase 1 — CLI Interface & User Experience

1.1

Build the Chat Terminal UI

Library: [Ink](#) — React-based CLI rendering. Production-tested, used by many dev tools.

```
// src/cli/App.tsx (conceptual)
import { render, Box, Text } from 'ink';
import { InputBar } from './InputBar';
import { OutputArea } from './OutputArea';
import { useAgentLoop } from '../agent/useAgentLoop';

export function App() {
  const { messages, isProcessing, sendMessage } = useAgentLoop();

  return (
    <Box flexDirection="column" height="100%">
      <Box flexGrow={1}>
        <OutputArea messages={messages} />
      </Box>
      <InputBar onSubmit={sendMessage} disabled={isProcessing} />
    </Box>
  );
}
```

Key Features to Implement:

- Persistent input bar at bottom (always ready for user input)
- Color-coded message types — user (green), assistant (blue), tool output (yellow), errors (red)
- Spinner / status indicator when agent is processing
- Scroll-back for conversation history

- Rich text formatting — code blocks with syntax highlighting, bold labels

Resources: [Ink GitHub](#) — [Ink Docs](#) — [Chalk](#) (colors) — [cli-spinners](#) (loading spinners)

1.2

Streaming & Real-Time Output

Why streaming matters: The agent makes multiple tool calls before responding. Without streaming, the user stares at a blank terminal for seconds. Streaming shows progress in real time — "Analyzing...", "Executing tool...", partial responses.

```
// Conceptual streaming integration
async function streamResponse(prompt: string, onChunk: (chunk: string) => void) {
  const stream = await provider.chatStream([
    { role: 'user', content: prompt }
  ])

  for await (const chunk of stream) {
    onChunk(chunk.content); // Updates OutputArea in real time
  }
}
```

Implementation Plan:

- Use `AsyncIterable` from LLM provider SDKs (OpenAI supports streaming natively)
- Pipe chunks into Ink's `useInput` / state updates for real-time rendering
- Show intermediate status updates: "🔍 Searching codebase...", "⚙️ Running tool: readFile"
- Buffer small chunks and flush periodically for performance

Resources: [OpenAI Streaming](#) — [Anthropic Streaming](#)

Input Handling & Command Parsing

- **Multi-line input:** Support paste-friendly multi-line prompts (Alt+Enter for newline, Enter to submit)
- **Special commands:** `/help` , `/clear` , `/config` , `/exit`
- **History navigation:** Up/Down arrows cycle through previous prompts
- **Autocomplete:** Tab-complete for file paths and tool names (nice-to-have)

Resources: [Inquirer](#) (alternative prompt library) — [Ink Input Example](#)

Phase 2 — Agentic Loop (ReAct Pattern)

2.1

Agent Loop Engine

Pattern: [ReAct \(Reasoning + Acting\)](#) — the agent alternates between reasoning about what to do and executing tool calls.

```
// src/agent/AgentLoop.ts (conceptual)
export class AgentLoop {
  async run(userInput: string): Promise<string> {
    const context = await this.contextBuilder.build(userInput);
    const history = this.memory.getRecentEpisodes(10);

    let done = false;
    let finalResponse = '';

    while (!done) {
      const response = await this.llm.chat([
        { role: 'system', content: this.buildSystemPrompt(context) },
        ...history,
        { role: 'user', content: userInput },
      ]);

      const action = this.responseParser.parseAction(response);

      if (action.type === 'final_answer') {
        finalResponse = action.content;
        done = true;
      } else if (action.type === 'tool_call') {
        const result = await this.toolExecutor.execute(action.toolCall);
        this.memory.addEpisode({ type: 'tool_call', summary: `Tool call: ${action.toolCall}` });
        // Feed result back to LLM for next reasoning step
        history.push({ role: 'tool', content: JSON.stringify(result) });
      }
    }

    return finalResponse;
  }
}
```

```

    return finalResponse;
  }
}

```

Loop Flow:

1. Receive user input → Build context (retrieve relevant code, memory)
2. Call LLM with system prompt + context + conversation history
3. Parse LLM response → either **tool_call** or **final_answer**
4. If tool_call → execute tool → feed observation back → go to step 2
5. If final_answer → return to user

Resources: [ReAct Paper \(Google DeepMind\)](#) — [ReAct Prompting Guide](#)

2.2

LLM Provider Abstraction (Adapter Pattern)

Pattern: Adapter — define a common interface so you can swap providers (OpenAI ↔ Anthropic ↔ local models) without changing agent logic.

```

// src/agent/providers/IProvider.ts
export interface ILLMProvider {
  chat(messages: Message[], options?: ChatOptions): Promise<ChatResponse>
  chatStream(messages: Message[], options?: ChatOptions): AsyncIterable<ChatResponse>
}

// src/agent/providers/OpenAIProvider.ts
export class OpenAIProvider implements ILLMProvider {
  constructor(private apiKey: string, private model: string = 'gpt-4o') {}

  async chat(messages: Message[], options?: ChatOptions): Promise<ChatResponse> {
    // Call OpenAI API, return formatted response
  }
}

```

```

    async *chatStream(messages: Message[], options?: ChatOptions) {
        // Stream from OpenAI, yield chunks
    }
}

// src/agent/providers/AnthropicProvider.ts
export class AnthropicProvider implements ILLMProvider {
    // Same interface, different implementation
}

```

Resources: [OpenAI SDK](#) — [Anthropic SDK](#) — [Adapter Pattern Guide](#)

2.3

Response Parsing & Structured Output

The LLM must respond in a predictable structured format (JSON) so your code can reliably parse tool calls vs final answers.

```

// Expected LLM response format
{
  "type": "tool_call",          // or "final_answer"
  "tool": "read_file",         // only for tool_call
  "arguments": {
    "path": "./src/index.ts"
  },
  "thought": "I need to read the entry point first",
  "id": "call_abc123"          // unique call ID
}

// OR for final answers:
{
  "type": "final_answer",
  "content": "The main entry point is `src/index.ts` which initi
}

```

Implementation tips:

- Use [Zod](#) to validate the parsed JSON structure
- Handle malformed JSON gracefully — retry with a correction prompt
- Use `response_format: { type: "json_object" }` on OpenAI for guaranteed JSON

Resources: [Zod Schema Validation](#) — [OpenAI Structured Outputs](#)

Phase 3 — Tool Registry & Tool Executor

3.1

Tool Registry (Factory / Service Locator Pattern)

Purpose: Tools register themselves and are resolved dynamically by name.

New tools can be added without modifying the agent loop.

```
// src/tools/registry/ToolRegistry.ts
export class ToolRegistry {
  private tools = new Map<string, Tool>();

  register(tool: Tool): void {
    this.tools.set(tool.name, tool);
  }

  registerMany(tools: Tool[]): void {
    tools.forEach(t => this.register(t));
  }

  get(name: string): Tool {
    const tool = this.tools.get(name);
    if (!tool) throw new Error(`Tool "${name}" not found`);
    return tool;
  }

  getAllDefinitions(): ToolDefinition[] {
    return Array.from(this.tools.values()).map(t => ({
      name: t.name,
      description: t.description,
      inputSchema: t.parameters,
    }));
  }
}
```

Key Decisions:

- **Dynamic loading:** Tools can be loaded from plugins/packages at runtime (useful for user extensions)
- **Registration at startup:** Scan a `tools/` directory and auto-register
- **Versioning:** Each tool can have a version — helpful for updates

Resources: [Factory Pattern](#) — [Service Locator Pattern](#)

3.2

Tool Executor (Command Pattern)

Pattern: Command — each tool execution is encapsulated as a command object with boundaries (timeout, output limits).

```
// src/tools/executor/ToolExecutor.ts
export interface ExecutionOptions {
  timeout: number;           // ms (e.g., 30000 = 30s)
  maxOutputBytes: number;    // (e.g., 1MB)
  workingDirectory?: string;
}

export class ToolExecutor {
  constructor(
    private registry: ToolRegistry,
    private options: ExecutionOptions
  ) {}

  async execute(toolName: string, args: Record<string, unknown>) {
    const tool = this.registry.get(toolName);

    // Pre-execution guardrails
    this.preValidator.validate(tool, args);

    // Execute with timeout via AbortController
    const controller = new AbortController();
    const timer = setTimeout(() => controller.abort(), this.opt
```

```

    try {
      const result = await tool.execute(args);

      // Post-execution guardrails
      this.postValidator.validate(result);

      // Cap output size
      if (this.exceedsMaxSize(result)) {
        result.data = this.truncate(result.data, this.options.maxOutputSize);
        result.truncated = true;
      }

      return result;
    } catch (error) {
      if (error.name === 'AbortError') {
        return { success: false, error: 'Tool execution timed out' };
      }
      return { success: false, error: error.message };
    } finally {
      clearTimeout(timer);
    }
  }
}

```

Critical Safety Features:

- **Timeouts:** Every tool execution gets an `AbortController`-based timeout. Prevents infinite loops.
- **Output capping:** Limit stdout/stderr to a max size. Prevents memory overflow from noisy commands.
- **Isolation:** Run in `child_process` — not the main thread. A crash in the tool doesn't crash the agent.

Resources: [Node.js child_process](#) — [AbortController](#) — [Command Pattern](#)

Security & Sandboxing

Two-tier approach:

- **Tier 1 (Default):** Spawn child processes with strict boundaries — timeout, output cap, allowed commands whitelist
- **Tier 2 (Future/Enterprise):** Docker container sandbox — each tool execution spins up a disposable container

```
// Example: Whitelist-based shell tool
export class ShellTool implements Tool {
  name = 'execute_shell';
  description = 'Run a shell command';
  parameters = { command: { type: 'string', description: 'The command to execute' } };

  private ALLOWED_COMMANDS = ['ls', 'cat', 'grep', 'find', 'node'];

  async execute(args: { command: string }): Promise<ToolResult> {
    const baseCmd = args.command.split(' ')[0];
    if (!this.ALLOWED_COMMANDS.includes(baseCmd)) {
      return { success: false, error: `Command "${baseCmd}" is not allowed` };
    }

    // Spawn child process with timeout
    const { exec } = require('child_process');
    return new Promise((resolve) => {
      const proc = exec(args.command, { timeout: 10000, maxBuffer: 1024 * 1024 });
      (error, stdout, stderr) => {
        if (error) return resolve({ success: false, error: error.message });
        resolve({ success: true, data: stdout });
      }
    });
  }
}
```

Docker Sandbox (Future):

```
// Conceptual Docker-based executor
docker run --rm --read-only \
  --memory="256m" --cpus="1" \
  --network=none \
  -v /tmp/workspace:/workspace:ro \
  node:20-alpine node /workspace/script.js
```

Resources: [Docker Security](#) — [Firecracker MicroVM](#) (for micro-VM approach)

3.4

Built-in & Custom Tools

Tool	Description	Example Usage
<code>read_file</code>	Read file contents with line ranges	Read <code>src/index.ts</code> lines 10–50
<code>write_file</code>	Write/save code changes (with confirmation)	Apply a bug fix to <code>utils.ts</code>
<code>search_code</code>	Grep/search across the codebase	Find all usages of <code>ToolRegistry</code>
<code>run_test</code>	Run a specific test or test suite	Run <code>pnpm test -- --grep "AgentLoop"</code>
<code>shell</code>	Execute a shell command (whitelisted)	<code>ls -la src/</code>
<code>list_files</code>	List directory contents	List all files in <code>src/tools/</code>

`get_stack_info`

Get project tech stack
config

Retrieve TypeScript version,
framework info

Phase 4 — Context Building & Retrieval

4.1

AST Parsing (TypeScript Compiler API)

Key insight: You *don't* need an LLM to understand code structure. Use the TypeScript Compiler API (or Babel for JS) to parse code into AST and extract functions, classes, interfaces, and their associated comments.

```
// src/context/parser/CodeParser.ts (conceptual)
import ts from 'typescript';

export interface CodeChunk {
  id: string;
  filePath: string;
  kind: 'function' | 'class' | 'interface' | 'variable';
  name: string;
  signature: string;
  docComment: string;
  code: string;           // The actual source text
  startLine: number;
  endLine: number;
}

export class CodeParser {
  parseFile(filePath: string): CodeChunk[] {
    const sourceFile = ts.createSourceFile(
      filePath, fs.readFileSync(filePath, 'utf8'),
      ts.ScriptTarget.Latest, true
    );
    const chunks: CodeChunk[] = [];

    ts.forEachChild(sourceFile, node => this.walk(node, chunks,

    return chunks;
  }
}
```

```

private walk(node: ts.Node, chunks: CodeChunk[], filePath: string) {
  if (ts.isFunctionDeclaration(node) && node.name) {
    chunks.push({
      id: `${filePath}:${node.name.text}`,
      filePath,
      kind: 'function',
      name: node.name.text,
      signature: node.getText().split('{')[0].trim(),
      docComment: this.getJSDoc(node),
      code: node.getText(),
      startLine: node.getStartLineOfPosition(sourceFile),
      endLine: node.getEndLineOfPosition(sourceFile),
    });
  }
  ts.forEachChild(node, child => this.walk(child, chunks, filePath));
}
}

```

What to extract from each chunk:

- **Identifier:** Unique ID (filePath + function/class name)
- **Signature:** The function/class signature (params, return type)
- **Doc comments:** JSDoc or preceding comments
- **Code body:** The full source text (for context)
- **Metadata:** File path, line range, last modified timestamp

Resources: [TypeScript Compiler API](#) — [TS AST Viewer](#) (interactive playground) — [Babel Parser](#) (for JS projects)

4.2

Embeddings Generation

Convert code chunks into vector embeddings for semantic search.

```
// src/context/embeddings/EmbeddingService.ts
export class EmbeddingService {
  constructor(
    private provider: 'openai' | 'local',
    private model: string = 'text-embedding-3-small' // OpenAI
  ) {}

  async embed(texts: string[]): Promise<number[][]> {
    if (this.provider === 'openai') {
      const resp = await openai.embeddings.create({
        model: this.model,
        input: texts,
      });
      return resp.data.map(d => d.embedding);
    }

    if (this.provider === 'local') {
      // Use sentence-transformers via onnx or local inference
      // HuggingFace's Transformers.js for Node.js
      return this.localEmbed(texts);
    }
  }
}
```

Approach	Pros	Cons
OpenAI Embeddings API <code>text-embedding-3-small</code>	High quality, cheap (\$0.02/1M tokens), no local compute	Requires internet, API cost
Local (Transformers.js) <code>sentence-transformers/all-MiniLM-L6-v2</code>	Offline, free, private	Slower, larger memory footprint
Cohere Embed	Good quality, simple API	Requires internet, API key

Resources: [OpenAI Embeddings](#) — [Sentence-Transformers](#) — [Transformers.js](#)
(local inference in Node)

4.3

Vector Database Setup

Local vector storage options for a CLI tool:

Option	Description	Best For
FAISS	Facebook's similarity search library. In-memory index, very fast.	Pure vector search, no persistence needed between sessions
sqlite-vec	SQLite extension for vector search. Persisted to disk.	Persistent storage + SQL querying
HNSWlib	Hierarchical Navigable Small World graphs. Fast approximate nearest neighbors.	Large indices where speed matters
LMDB + Flat vectors	Memory-mapped key-value store with flat vector files.	Zero-dependency, simple approach

```
// src/context/vector-store/VectorStore.ts (conceptual)
export interface VectorStore {
  insert(id: string, vector: number[], metadata: Record<string, any>): Promise<void>;
  update(id: string, vector: number[], metadata: Record<string, any>): Promise<void>;
  delete(id: string): Promise<void>;
  search(query: number[], topK: number): Promise<SearchResult[]>;
}

export interface SearchResult {
  id: string;
  score: number; // similarity score (cosine distance)
```

```
metadata: Record<string, unknown>;  
}
```

Resources: [FAISS GitHub](#) — [sqlite-vec](#) — [HNSWlib](#)

4.4

Incremental Index Updates

Problem: When a file changes, you can't dump and rebuild the entire index. You need incremental updates.

Solution: Watch for file changes using `fs.watch` / `chokidar`, re-parse only the changed file, re-embed changed chunks, and update/delete vectors by ID.

```
// Incremental update flow  
class IndexManager {  
  async onFileChange(filePath: string) {  
    // 1. Parse only this file  
    const newChunks = this.parser.parseFile(filePath);  
  
    // 2. For each chunk, check if it changed (compare hash)  
    for (const chunk of newChunks) {  
      const existingHash = await this.getStoredHash(chunk.id);  
      const newHash = this.hash(chunk.code);  
  
      if (newHash !== existingHash) {  
        // 3. Re-embed and update  
        const vector = await this.embeddings.embed([chunk.code])  
        await this.vectorStore.update(chunk.id, vector[0], {  
          filePath: chunk.filePath,  
          name: chunk.name,  
          kind: chunk.kind,  
          signature: chunk.signature,  
          timestamp: Date.now(),  
        });  
        await this.storeHash(chunk.id, newHash);  
      }  
    }  
  }  
}
```

```

    }
  }

  // 4. Handle deletions (chunks removed from file)
  const removedChunks = await this.findRemovedChunks(filePath)
  for (const id of removedChunks) {
    await this.vectorStore.delete(id);
  }
}
}

```

Resources: [Chokidar](#) (file watcher) — [fs.watchFile](#)

4.5

Query-Time Retrieval & Ranking

```

// src/context/retrieval/Retriever.ts
export class Retriever {
  async retrieve(query: string, topK: number = 10): Promise<Ret
    // 1. Embed the user query
    const queryVector = await this.embeddings.embed([query]);

    // 2. Search vector DB by semantic similarity
    let results = await this.vectorStore.search(queryVector[0],

    // 3. Apply metadata filters (context cues)
    if (query.match(/frontend|ui|component/i)) {
      results = results.filter(r =>
        r.metadata.filePath?.includes('components/') ||
        r.metadata.filePath?.includes('ui/') ||
        r.metadata.filePath?.endsWith('.tsx')
      );
    }

    if (query.match(/recent|latest|new|last/i)) {
      results = results.sort((a, b) =>

```

```

        (b.metadata.timestamp as number) - (a.metadata.timestamp
    );
}

// 4. Rank and return top K
return results.slice(0, topK).map(r => ({
    id: r.id,
    code: this.loadOriginalCode(r.id),
    score: r.score,
    metadata: r.metadata,
}));
}
}

```

Context Cues (Metadata Filtering):

- **File path:** Filter by module/folder (e.g., "frontend", "backend")
- **Timestamp:** Sort/filter by recency when query implies "recent"
- **Kind:** Filter by function/class/interface type
- **Module name:** Match against known project modules

Resources: [Vector Similarity Explained](#) — [Cosine Similarity](#)

Phase 5 — Memory Architecture

5.1

Working Memory (In-Memory)

What it holds: Current conversation state — the active user query, the last N turns, current context snippets, active tool results.

```
// src/memory/WorkingMemory.ts
export class WorkingMemory {
  private turns: Message[] = [];
  private maxTurns: number = 20; // Limit to avoid token over

  addTurn(message: Message): void {
    this.turns.push(message);
    if (this.turns.length > this.maxTurns) {
      this.turns.shift(); // Evict oldest
    }
  }

  getRecentTurns(count: number = 10): Message[] {
    return this.turns.slice(-count);
  }

  clear(): void {
    this.turns = [];
  }
}
```

Working Memory is NOT persisted. It lives only during the agent session. When the CLI exits, it's gone — unless key moments were saved to episodic memory.

Episodic Memory (SQLite)

What it holds: Structured records of important events across sessions — tool calls made, files modified, decisions taken, errors encountered.

You *don't* store every LLM response. You distill interactions into structured entries at the moment they happen.

```
// src/memory/EpisodicMemory.ts
import Database from 'better-sqlite3';

export class EpisodicMemory {
  private db: Database.Database;

  constructor(dbPath: string) {
    this.db = new Database(dbPath);
    this.db.exec(`
      CREATE TABLE IF NOT EXISTS episodes (
        id TEXT PRIMARY KEY,
        timestamp INTEGER NOT NULL,
        type TEXT NOT NULL CHECK(type IN ('tool_call','decision')),
        summary TEXT NOT NULL,
        details TEXT    -- JSON blob for extra data
      )
    `);
  }

  async add(entry: EpisodicEntry): Promise<void> {
    this.db.prepare(
      'INSERT INTO episodes (id, timestamp, type, summary, details) VALUES (?, ?, ?, ?, ?)'
    ).run(entry.id, entry.timestamp, entry.type, entry.summary,
      entry.details ? JSON.stringify(entry.details) : null)
  }

  async getRecent(limit: number = 20, types?: string[]): Promise<EpisodicEntry[]> {
    let query = 'SELECT * FROM episodes';
    if (types?.length) {
      query += ` WHERE type IN (${types.map(() => '?').join(',')})`;
    }
  }
}
```

```

    }
    query += ' ORDER BY timestamp DESC LIMIT ?';

    return this.db.prepare(query).all(...(types || []), limit) as T[]
  }

  async pruneOlderThan(timestamp: number): Promise<void> {
    this.db.prepare('DELETE FROM episodes WHERE timestamp < ?')
      .run(timestamp)
  }
}

// Event structure for what gets stored:
// { type: 'tool_call', summary: 'read_file: src/index.ts', details: '...' }
// { type: 'decision', summary: 'Selected OpenAIModel for this step', details: '...' }
// { type: 'file_change', summary: 'Modified src/agent/AgentLoop.ts', details: '...' }

```

Key Design Decisions:

- **Don't store raw LLM responses** — they're verbose. Store summaries and outcomes.
- **Log as you go** — when a tool call completes, immediately save a structured entry. No post-hoc LLM summarization needed.
- **Prune periodically** — old episodes get archived/deleted to keep the DB lean.

Resources: [SQLite](#) — [better-sqlite3](#) (sync, fast, no ORM needed) — [WAL mode](#) (for concurrent reads)

5.3

Procedural Memory (Config File)

What it holds: The project's tech stack, framework versions, constraints, and environment settings. This ensures the LLM generates code compatible with the actual project setup, not assumed latest versions.

```

// src/memory/ProceduralMemory.ts
// Stored as a JSON config file alongside the project

// config/project-config.json
{
  "stack": {
    "typescript": "5.4.5",
    "node": "20.11.0",
    "react": "18.3.1",
    "next": "14.2.0",
    "vitest": "1.6.0"
  },
  "constraints": [
    "Must use pnpm (not npm)",
    "Must not use class components – only functional components",
    "ESLint rules: no-any, strict equality"
  ],
  "lastUpdated": 1747267200000
}

// Usage: Load at startup, inject into system prompt
const config = new ProceduralMemory('./config/project-config.json')

function buildSystemPrompt(context: string): string {
  return `You are an AI coding assistant for this project.

Project Stack:
${JSON.stringify(config.getStack(), null, 2)}

Constraints:
${config.getConstraints().join('\n')}

Project Context:
${context}
`;
}

```

Maintenance:

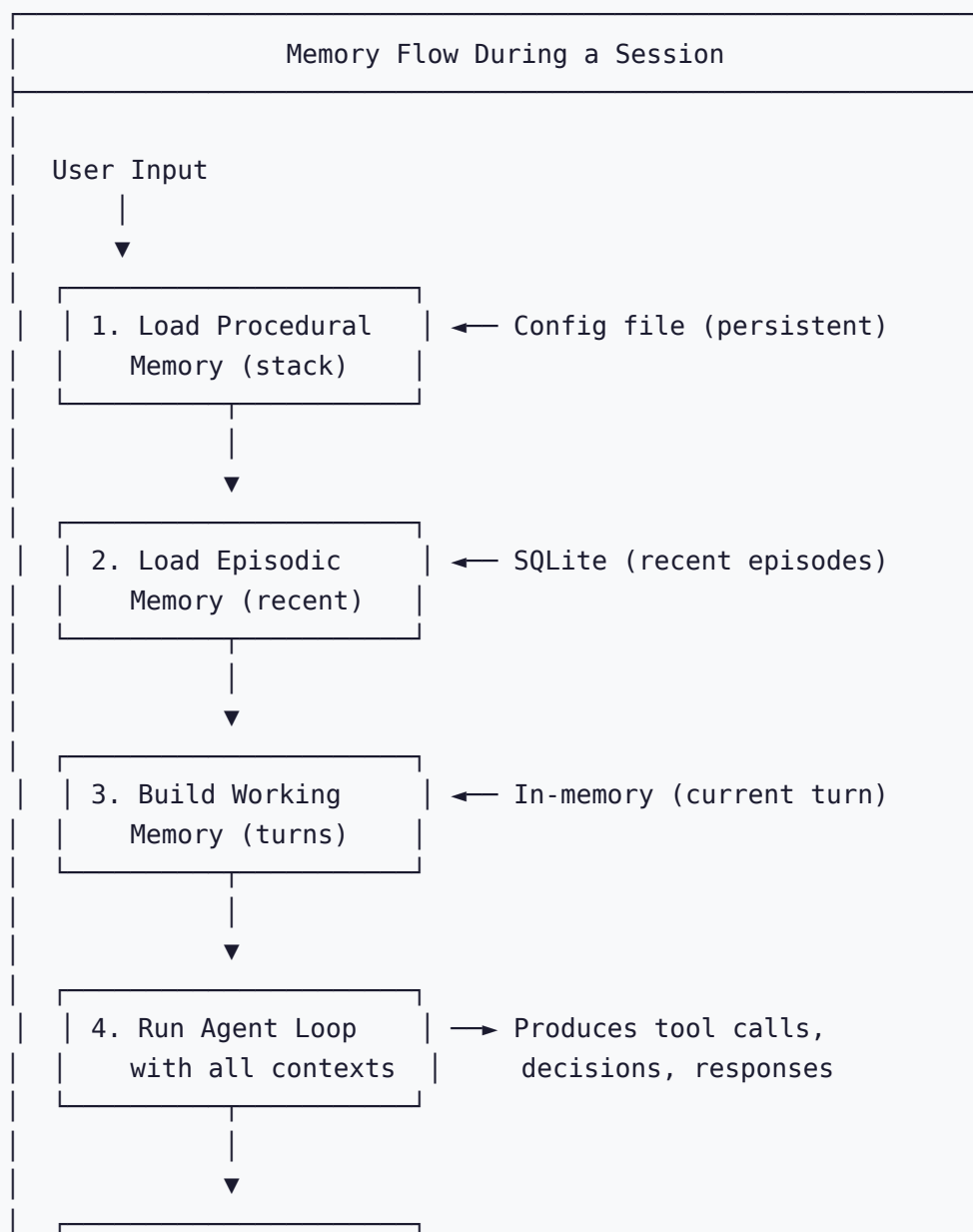
- Provide commands: `/config set stack.typescript 5.5.0`

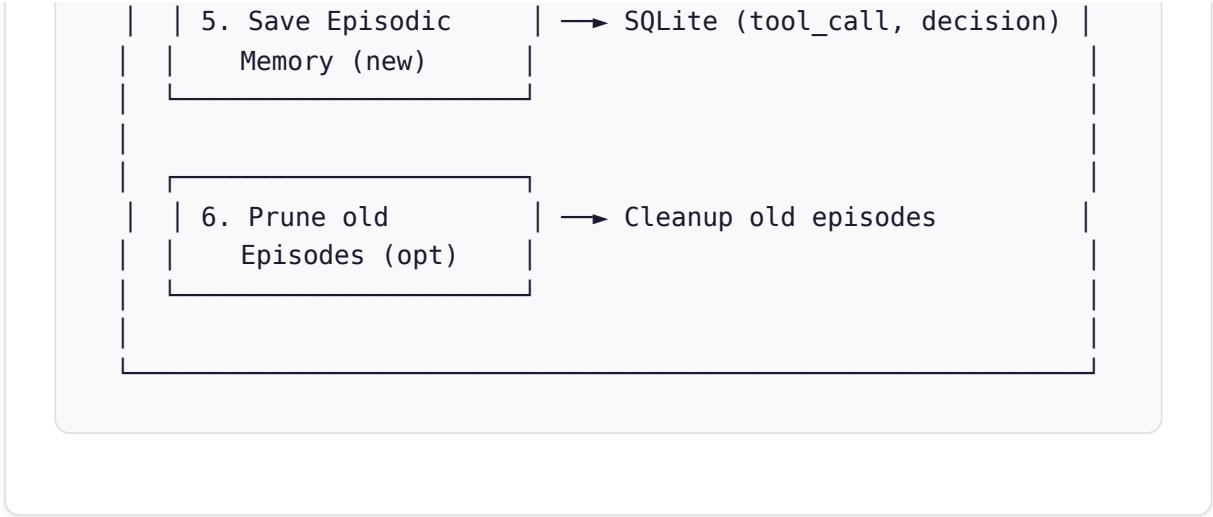
- Auto-detect: Scan `package.json` / `requirements.txt` on init
- Periodic checks: Remind user when versions drift

Resources: [node-config](#) — [node-config GitHub](#)

5.4

Memory Integration & Lifecycle





Phase 6 — Guardrails & Error Handling

6.1

Pre-Call Validation

Before executing any tool, validate inputs to catch malformed or dangerous requests.

```
// src/guardrails/PreCallValidator.ts
export class PreCallValidator {
  validate(tool: Tool, args: Record<string, unknown>): void {
    // 1. Required parameters check
    for (const [name, param] of Object.entries(tool.parameters)) {
      if (param.required && args[name] === undefined) {
        throw new Error(`Missing required parameter: ${name}`);
      }
    }

    // 2. Type validation
    for (const [name, value] of Object.entries(args)) {
      const schema = tool.parameters[name];
      if (schema && typeof value !== schema.type) {
        throw new Error(`Parameter "${name}" expected ${schema.type}`);
      }
    }

    // 3. Security: No path traversal
    if (args.path && typeof args.path === 'string') {
      if (args.path.includes('.') || args.path.startsWith('/')) {
        throw new Error('Path traversal detected: access denied');
      }
    }

    // 4. Size limits on inputs
    if (args.command && typeof args.command === 'string' && args.command.length > 1000) {
      throw new Error('Command too long (max 1000 characters)');
    }
  }
}
```

```

    }
  }
}

```

6.2

Post-Call Validation

After tool execution, validate the output before feeding it back to the LLM.

```

// src/guardrails/PostCallValidator.ts
export class PostCallValidator {
  validate(result: ToolResult): void {
    // 1. Output size check
    if (result.data && typeof result.data === 'string') {
      if (result.data.length > 1024 * 100) { // 100KB
        result.data = result.data.slice(0, 1024 * 100) + '\n...'
        result.truncated = true;
      }
    }

    // 2. Sensitive data detection (basic)
    if (result.data && typeof result.data === 'string') {
      const sensitivePatterns = [
        /-----BEGIN (RSA|EC|OPENSSH) PRIVATE KEY-----/,
        /sk-[a-zA-Z0-9]{20,}/, // OpenAI API key pattern
        /ghp_[a-zA-Z0-9]{36}/, // GitHub token pattern
        /(?:https?:\/\/\/)?[^\s]+\.(?:env|pem|key)\b/,
      ];
      for (const pattern of sensitivePatterns) {
        if (pattern.test(result.data)) {
          result.data = '[SENSITIVE DATA REDACTED]';
          break;
        }
      }
    }

    // 3. Error detection (stderr always captured)

```

```

    // Already handled by ToolExecutor
  }
}

```

6.3

Output Guardrails

Validate the LLM's response before acting on it.

```

// src/guardrails/OutputGuard.ts
export class OutputGuard {
  check(response: LLMResponse): GuardResult {
    const issues: string[] = [];

    // 1. JSON parseable?
    if (!this.isValidJson(response)) {
      issues.push('Response is not valid JSON');
    }

    // 2. Tool exists?
    if (response.type === 'tool_call') {
      if (!this.toolRegistry.has(response.tool)) {
        issues.push(`Tool "${response.tool}" does not exist`);
      }

      // 3. Arguments match schema?
      const tool = this.toolRegistry.get(response.tool);
      for (const key of Object.keys(response.arguments)) {
        if (!tool.parameters[key]) {
          issues.push(`Unknown parameter "${key}" for tool "${response.tool}"`);
        }
      }
    }

    // 4. Content safety (no code injection via prompt)
    if (typeof response.content === 'string') {
      if (this.containsInjectionPatterns(response.content)) {

```

```

        issues.push('Response contains potential injection pattern')
    }
}

return { valid: issues.length === 0, issues };
}
}

```

6.4

Recovery & Fallback Strategies

Situation	Recovery Strategy
LLM returns malformed JSON	Auto-retry with a correction prompt: "Your response was not valid JSON. Please respond with valid JSON only." (max 2 retries)
Tool execution fails	Log the error to episodic memory, inform the LLM and ask it to try a different approach or tool
Tool execution times out	Kill the process, return timeout error to LLM, ask to try a simpler command
LLM calls non-existent tool	Log as guardrail violation, respond with available tools list, ask to retry
File not found	Return clear error, suggest related files that do exist
Embedding service down	Fall back to keyword/grep-based search (no embeddings)

Key principle: Never crash. Every failure should produce a clear message and a path forward — either automatic retry or user clarification request.

Phase 7 — Edge Cases & Their Handling

7.1 Infinite Loops / Runaway Processes

Scenario: LLM asks to run `node run.ts` where `run.ts` has an infinite loop (`while(true) {}`).

Handling:

- **Timeouts on every tool call** — AbortController-based timeout on `child_process`. Default: 30s.
- **Configurable per tool** — file reads can have shorter timeout (5s), shell commands longer (60s).
- **Process tree kill** — ensure child and all its sub-processes are killed on timeout.

```
// Kill entire process tree on timeout
import { execSync } from 'child_process';

function killProcessTree(pid: number) {
  try {
    execSync(`kill -- -$(ps -o pgid= -p ${pid} | tr -d ' ')\`, { t
  } catch {
    process.kill(pid, 'SIGKILL'); // Force kill
  }
}
```

7.2 Excessive Output from Tools

Scenario: A tool (e.g., running a test suite or cat'ing a large file) prints megabytes of data to stdout.

Handling:

- **Output byte limit:** Cap at 1MB by default. Use `maxBuffer` option in `exec`.
- **Streaming truncation:** For long-running output, stream chunk by chunk and stop after limit.
- **Flag as truncated:** Set `truncated: true` in `ToolResult` so the LLM knows it didn't get full output.
- **Summary mode:** For very large outputs, prepend a summary: `"[Output truncated at 1MB. First 100 lines shown...]"`

7.3 Malicious Code Execution

Scenario: User's project contains malicious code, or the LLM is prompted to run dangerous commands (`rm -rf /` , `curl | bash`).

Handling:

- **Command whitelist:** Only allow safe commands (`ls`, `cat`, `grep`, `find`, `node`, `npm`, `git`). Block shell pipes `|` , redirects `>` , `rm` , `sudo` , `curl` , `wget` .
- **Path restriction:** Limit file operations to the project directory. Block `..` and absolute paths above project root.
- **Read-only by default:** File reads are allowed by default. Writes require explicit user confirmation.
- **Docker sandbox (future):** For full isolation, run each tool in a disposable Docker container.

7.4 Stale Embeddings / File Changes

Scenario: User modifies a file. The vector DB still has old embeddings, so context retrieval returns outdated code.

Handling:

- **File watcher:** Use `chokidar` to detect file saves. On change, re-parse that file, re-embed changed functions, update by ID.

- **Hash-based change detection:** Store a hash of each chunk's code. Only re-embed if hash changed.
- **Startup sync:** On CLI launch, check all file modification timestamps and re-index any changed files.
- **Graceful fallback:** If vector DB is stale, also do a fresh file read before feeding context to LLM.

7.5 LLM Returns Irrelevant Responses

Scenario: LLM goes off-topic, calls wrong tools, or gives nonsensical code suggestions.

Handling:

- **Structured output schema:** Force JSON with `response_format` (OpenAI) or structured tool use (Anthropic).
- **Validate tool existence:** Before executing, check the tool is registered. If not, respond with available tools list.
- **Lint/syntax check:** Before applying code suggestions, run TypeScript compiler check on the proposed code.
- **Clarity check:** If response is vague, auto-prompt: "Please be more specific about which file and what change."
- **Max retries:** Limit the loop to N iterations (e.g., 15 tool calls max) to prevent infinite agent loops.

7.6 User Input Variability (Synonyms & Fuzzy Queries)

Scenario: User says "fix the recent bug in the UI" — how does the system know "recent" means "last modified"?

Handling:

- **Semantic matching:** The word "recent" gets embedded and its vector is close to "new", "latest", "last". Embedding captures the *meaning*.

- **Metadata filtering:** When query meaning relates to time, apply timestamp-based sorting/filtering in the retrieval step.
- **Your code interprets:** After retrieving semantic results, your code checks if the query implies time-based filtering and applies it. No LLM needed for this step.
- **Combined approach:** Semantic similarity finds *what* is relevant. Metadata filtering refines *which* specific item is relevant.

7.7 Large Conversation History

Scenario: After 50+ turns, the conversation history is too large for the LLM context window.

Handling:

- **Sliding window:** Only keep the last N turns (e.g., 10-20) in working memory.
- **Episodic summarization:** Store key outcomes in SQLite (not raw responses). Retrieve relevant episodes when needed.
- **Token budget:** Track token usage per turn. When approaching limit, summarize old turns into a single system message.
- **Structured summary:** "Earlier in the conversation: modified X file, ran Y tool, decided on Z approach."

Phase 8 — Testing, Deployment & Maintenance

8.1

Testing Strategy

Layer	What to Test	Tools
Unit Tests	Individual tools, validators, parsers, memory operations	Vitest
Integration Tests	Tool registry + executor flow, context builder pipeline, memory lifecycle	Vitest + test fixtures (sample projects)
Agent Loop Tests	Mock LLM provider, verify correct tool calls and final answer	Vitest + mocks
E2E Tests	Full user session on a sample project	Custom test harness + sample repo
Security Tests	Path traversal attempts, command injection, timeout enforcement	Vitest + adversarial test cases

```
// Example: Integration test for Tool Executor
describe('ToolExecutor', () => {
  it('should timeout on infinite loops', async () => {
    const executor = new ToolExecutor(registry, { timeout: 1000 })
    const result = await executor.execute('shell', { command: 'cat /dev/null' })
    expect(result.success).toBe(false);
    expect(result.error).toContain('timed out');
  });

  it('should truncate excessive output', async () => {
    const executor = new ToolExecutor(registry, { maxOutputBytes: 100 })
    const result = await executor.execute('shell', { command: 'cat /dev/null' })
    expect(result.truncated).toBe(true);
    expect(result.data.length).toBeLessThanOrEqual(100);
  });
});
```

```
});

it('should block dangerous commands', async () => {
  const executor = new ToolExecutor(registry);
  const result = await executor.execute('shell', { command: '
  expect(result.success).toBe(false);
  expect(result.error).toContain('not allowed');
});
});
```

8.2

Distribution & Updates

- **npm package:** Publish to npm registry. Users install via `npm install -g @yourname/cli-agent`.
- **Versioning:** Use semantic versioning (semver). Tools are included in the package, so updates come with `npm update -g @yourname/cli-agent`.
- **Plugin system (future):** Allow third-party tools via npm packages. Users install plugins and they auto-register.
- **Auto-update check:** On startup, ping npm registry to check for newer version. Notify user.

```
// Check for updates
async function checkForUpdates() {
  const current = require('./package.json').version;
  const resp = await fetch(`https://registry.npmjs.org/@yourname
  const { version: latest } = await resp.json();
  if (latest !== current) {
    console.log(`Update available: ${current} → ${latest}. Run:
  }
}
```

8.3

Logging & Observability

- **Structured logging:** Use [pino](#) for JSON-structured logs. Write to file (`~/.cli-agent/logs/`).
- **Log levels:** `debug` (tool calls), `info` (user messages), `warn` (validation warnings), `error` (failures).
- **Telemetry (opt-in):** Anonymous usage stats — which tools are used most, error rates.
- **Debug mode:** `--verbose` flag prints logs to terminal for troubleshooting.

8.4

Future Upgrades

Feature	Why	Effort
Docker Sandbox	Full execution isolation for untrusted code	High
Plugin System	Community-contributed tools as npm packages	Medium
Multi-Model Support	Allow user to choose between GPT-4, Claude, Gemini, local models	Medium (architected already via Adapter pattern)
GUI Mode	Electron/web-based interface alongside CLI	High

Team Collaboration	Shared memory/index across team members (episodic DB sync)	High
VS Code Extension	Integrate agent directly into editor	Medium
Custom Agent Personas	Let users define custom system prompts and tool sets per project	Low

Quick Reference: Libraries & Tools Summary

Category	Library / Tool	Install	Purpose
CLI UI	Ink	<code>npm install ink</code>	React-based terminal UI framework
Colors	Chalk	<code>npm install chalk</code>	Terminal string styling with colors
Spinners	cli-spinners	<code>npm install cli-spinners</code>	Loading spinners for terminal
AST Parsing	TypeScript	<code>npm install typescript</code>	TypeScript Compiler API for AST
Embeddings	openai	<code>npm install openai</code>	OpenAI API (embeddings + chat)

Local Embeddings	@xenova/transformers	<code>npm install @xenova/transformers</code>	Local inference for embeddings (no API key)
Vector Store	sqlite-vec	<code>npm install sqlite-vec</code>	SQLite-based vector search
Vector Store	faiss-node	<code>npm install faiss-node</code>	FAISS bindings for Node.js
Database	better-sqlite3	<code>npm install better-sqlite3</code>	SQLite for episodic memory
File Watcher	chokidar	<code>npm install chokidar</code>	File system change detection
Schema Validation	zod	<code>npm install zod</code>	Runtime schema validation for LLM responses
DI Container	tsyringe	<code>npm install tsyringe reflect-metadata</code>	Dependency injection (optional)
Logging	pino	<code>npm install pino</code>	Structured JSON logging
Testing	vitest	<code>npm install -D vitest</code>	Unit & integration testing
Linting	ESLint	<code>npm install -D eslint</code>	Code quality & consistency

Formatter	Prettier	<code>npm install -D prettier</code>	Code formatting
-----------	--------------------------	--------------------------------------	-----------------

Resources Grouped by Phase

Phase	Resources
Phase 0: Setup	TypeScript Project Setup · tsup (bundler) · Vitest · pnpm
Phase 1: CLI UI	Ink · Ink Docs · Chalk · cli-spinners · Inquirer
Phase 2: Agent Loop	ReAct Paper · ReAct Prompting Guide · OpenAI SDK · Anthropic SDK · Adapter Pattern · Zod · OpenAI Structured Outputs
Phase 3: Tool System	Node.js child_process · Command Pattern · Factory Pattern · Service Locator · Docker Security · Firecracker MicroVM · tsyringe (DI)
Phase 4: Context & Retrieval	TypeScript Compiler API · TS AST Viewer · Babel Parser · OpenAI Embeddings · Sentence-Transformers · Transformers.js · FAISS · sqlite-vec · Chokidar · Vector Similarity
Phase 5: Memory	SQLite · better-sqlite3 · WAL Mode · node-config
Phase 6: Guardrails	Zod · OWASP Top 10 · Anthropic Guardrails Guide
Phase 8: Testing & Deploy	Vitest · npm Packages · Semantic Versioning · pino logging

CLI Agent Project Guide — Generated from architectural discussion

Tech Stack: TypeScript 5.x · Node.js 20+ · Ink · SQLite · FAISS · Vitest